

C to C++

Inheritance

- involves deriving new classes from existing ones, known as the base class, to extend or modify their members.

Purpose

- ▶ Facilitates code and interface sharing
- ▶ Establishes a hierarchy of related types

Hierarchy in OOP

Manages Complexity

Hierarchy organizes objects into classifications, enabling better management of complexity.

Example: Periodic Table

- ▶ Elements share certain properties
- ▶ Elements classified as gases share certain additional properties.
- ▶ Inert gases represent a special class within gases.
- ▶ This hierarchical structure helps in understanding behaviors.

OOP Design Methodology

Steps

1. Decide on types.
2. Design relationships.
3. Use inheritance for code sharing.

Class Derivation Example

```
enum support { ta, ra, fellowship, other };
enum year { frsh, soph, junior, senior, grad };
class student {
private:
    int student_id;
    double gpa;
public:
    student(char* nm, int id, double g, year x);
    void print();
};

class grad_student : public student {
private:
    char dept[100];
    char thesis[80];
public:
    grad_student(char* nm, int id, double g, year x,
                 support t, char* d, char* th);
    void other();
};
```

Explanation

- ▶ `grad_student` is derived from the base class `student`.
- ▶ Public inheritance implies that public members of `student` become public members of `grad_student`.
- ▶ Private members of the base class cannot be accessed directly in the derived class.

Application

System Design

- ▶ Database for a college can be derived from the base class person.
- ▶ Student categories (e.g., graduate, undergraduate) can be further extended.
- ▶ Provides a comprehensive design for the system.

Polymorphism

Polymorphic Functions

- ▶ A polymorphic function has many forms.
- ▶ Example: Division operator in C.
- ▶ Behavior depends on argument types. Example: Integer division vs real division.

Polymorphism

- ▶ Polymorphism localizes responsibility for behavior.
- ▶ Client code usually requires no revision when additional functionality is added through ADT-provided code improvements.

struct shape in C

```
struct shape {  
    enum {CIRCLE, RECTANGLE} e_val;  
    double center, radius, height, width;  
};  
  
double area(shape* s) {  
    switch(s->e_val) {  
        case CIRCLE:  
            return (PI * s->radius * s->radius);  
        case RECTANGLE:  
            return (s->height * s->width);  
    }  
}
```

class shape in C++

```
class shape {
public:
    virtual double area() = 0; // pure virtual function
};

class rectangle : public shape {
private:
    double height, width;
public:
    rectangle(double h, double w) : height(h), width(w) {}
    double area() { return height * width }
};

class circle : public shape {
private:
    double radius;
public:
    circle(double r) : radius(r) {}
    double area() { return 3.14 * radius * radius }
};
```

Hierarchy in Object-Oriented Programming

- ▶ Inheritance mechanism used to create a hierarchy of related types.
- ▶ Example: Base class `shape` with derived classes `circle` and `rectangle`.
- ▶ Code improvements are localized in new derived classes.

Abstract Base Class

- ▶ Abstract base class (shape) contains pure virtual functions.
- ▶ Example: `virtual double area() = 0;`
- ▶ Definition provided in derived classes.

class square

```
class square : public rectangle {  
public:  
    square(double h) : rectangle(h, h) {}  
    double area() { return rectangle::area(); }  
};
```

Example: Square Class

- ▶ Derived class square inherits from rectangle.
- ▶ Code improvements localized in new derived class.
- ▶ Client code remains unchanged; not so in C.

Overloading

Definition

Overloading refers to defining multiple methods in a class with the same name but with different parameter lists.

- ▶ Methods can have different numbers or types of parameters.
- ▶ Compiler determines which method to call based on arguments' number and types.
- ▶ Overloading is compile-time (static) polymorphism.

Polymorphism

Definition

Polymorphism allows objects of different classes to be treated as objects of a common superclass.

- ▶ Achieved through method overriding.
- ▶ Method overriding is a form of run-time (dynamic) polymorphism.
- ▶ The correct method is invoked based on the actual object type at runtime.

Templates in C++

Parametric Polymorphism

C++ uses the keyword **template** to provide parametric polymorphism. Parametric polymorphism allows the same code to be used with respect to different types, where the type is a parameter of the code body.

An especially important use for this technique is in writing generic container classes. A container class is used to contain data of a particular type. Stacks, vectors, trees, and lists are all examples of standard container classes. We shall develop a stack container class as a parameterized type.

Templates in C++

Syntax of Template Declaration

The syntax of the class declaration is prefaced by:

```
template <class id>
```

This identifier `id` is a template argument that stands for an arbitrary type. Throughout the class definition, the template argument can be used as a type name. This argument is instantiated in the actual declarations.

Templates in C++

Parametric Polymorphism

A **template** in C++ provides **parametric polymorphism**, allowing the same code to be used with different types.

```
// Template definition for stack container class
template <class TYPE>
class stack {
public:
    stack(int size = 1000) :
        max_len(size) { s = new TYPE[size]; top = EMPTY; }
    ~stack() { delete[] s; }
    void reset() { top = EMPTY; }
    bool push(TYPE c) { s[++top] = c; }
    TYPE pop() { return s[top--]; }
    bool top_of() { return top == EMPTY; }
    bool empty() { return top == EMPTY; }
    bool full() { return top == max_len - 1; }
private:
    enum { EMPTY = -1 }; TYPE* s; int max_len; int top;
};
```

Templates in C++

Parametric Polymorphism (cont'd)

```
// Usage examples of stack template
stack<char> stk_ch;
    // Stack of characters
stack<char*> stk_str(200);
    // Stack of character pointers
stack<complex> stk_cmp(100);
    // Stack of complex numbers
```

Templates in C++

Parametric Polymorphism (cont'd)

```
// Reverse a series of char*
void reverse(char* str[], int n) {
    stack<char*> stk(n);
    for (int i = 0; i < n; ++i)
        stk.push(str[i]);
    for (int i = 0; i < n; ++i)
        str[i] = stk.pop();
}

// Initialize a stack of complex numbers from an array
void init(complex c[], stack<complex>& stk, int n) {
    for (int i = 0; i < n; ++i)
        stk.push(c[i]);
}
```

Benefits of OOP

Benefits of OOP

- ▶ The central element of OOP is the encapsulation of an appropriate set of data types and their operations.
- ▶ The **class** construct, with its member functions and data members, provides an appropriate coding tool.
- ▶ Class variables are the objects to be manipulated.
- ▶ Classes also provide **data hiding**. Access privileges can be managed and limited to whatever group of functions needs access to implementation details. This promotes modularity and robustness.

Benefits of Object-oriented Programming (cont'd)

- ▶ Another important concept in OOP is the promotion of code reuse through the **inheritance** mechanism.
- ▶ This mechanism involves deriving a new class from an existing one called the base class.
- ▶ A hierarchy of related data types can be created that share code.
- ▶ Many useful data structures are variants of one another, and without inheritance, minor variation would require code replication.

Benefits of Object-oriented Programming (cont'd)

- ▶ The OOP programming task is frequently more difficult than normal procedural programming as found in C.
- ▶ There is at least one extra design step before getting to the coding of algorithms involving the hierarchy of types.
- ▶ The solution will be more encapsulated, robust, easier to maintain and change.
- ▶ The solution will be more reusable.

Benefits of Object-oriented Programming (cont'd)

- ▶ All these benefits are especially important for large coding projects that require coordination among many programmers.
- ▶ Header files specifying general interfaces for different classes allow each programmer to work on individual code segments with a high degree of independence and integrity.
- ▶ OOP is many things to many people and can be summarized as:

OOP = type-extensibility + polymorphism

Pointers to Functions

Pointers to Functions

- ▶ In C, functions are not variables, but pointers to functions can be used.
- ▶ Pointers to functions can be assigned, placed in arrays, passed to functions, and returned by functions.
- ▶ We'll modify the sorting procedure to demonstrate the use of function pointers.

Pointers to Functions

```
#include <stdio.h>
#include <string.h>
#define MAXLINES 5000

char *lineptr[MAXLINES];

int readlines(char *lineptr[], int nlines);
void writelines(char *lineptr[], int nlines);
void qsort(void *lineptr[], int left, int right,
           int (*comp)(void *, void *));
int numcmp(char *, char *);

main(int argc, char *argv[]) {
    int nlines;
    int numeric = 0;

    if (argc > 1 && strcmp(argv[1], "-n") == 0)
        numeric = 1;
```

Pointers to Functions

```
if ((nlines = readlines(lineptr, MAXLINES)) >= 0) {  
    qsort((void**) lineptr, 0, nlines-1,  
          (int (*)(void*,void*))(numeric ? numcmp : strcmp));  
    writelines(lineptr, nlines);  
    return 0;  
} else {  
    printf("input too big to sort\n");  
    return 1;  
}  
}
```


Pointers to Functions

```
void qsort(void *v[], int left, int right,  
           int (*comp)(void *, void *)) {  
    int i, last;  
    void swap(void *v[], int, int);  
  
    if (left >= right)  
        return;  
  
    swap(v, left, (left + right)/2);  
    last = left;  
  
    for (i = left+1; i <= right; i++)  
        if ((*comp)(v[i], v[left]) < 0)  
            swap(v, ++last, i);  
  
    swap(v, left, last);  
    qsort(v, left, last-1, comp);  
    qsort(v, last+1, right, comp);  
}
```

Pointers to Functions

```
int numcmp(char *s1, char *s2) {  
    double v1, v2;  
    v1 = atof(s1);  
    v2 = atof(s2);  
  
    if (v1 < v2)  
        return -1;  
    else if (v1 > v2)  
        return 1;  
    else  
        return 0;  
}
```

Pointers to Functions

```
void swap(void *v[], int i, int j) {  
    void *temp;  
    temp = v[i];  
    v[i] = v[j];  
    v[j] = temp;  
}
```

Pointers to Functions

- ▶ Pointers to functions allow flexible handling of different sorting criteria.
- ▶ Function pointers can be passed to sorting algorithms, enabling sorting based on custom comparisons.
- ▶ The use of generic pointers (`void *`) enables handling different data types.

Miscellaneous Functions

Miscellaneous Functions

- ▶ The standard library offers a diverse set of functions.
- ▶ This section provides a brief overview of some useful functions.
- ▶ String operations, character class testing, ungetc, command execution, storage management, mathematical functions, and random number generation are covered.

String Operations

`strcat(s, t)`

`strncat(s, t, n)` concat n char of t to end of s

`strcmp(s, t)`

`strncmp(s, t, n)` compare first n char of s and t

`strcpy(s, t)`

`strncpy(s, t, n)` copy $\leq n$ char of t to s

`strlen(s)`

`strchr(s, c)` return ptr to first c in s

`strrchr(s, c)` return ptr to last c in s

- ▶ String functions for concatenation, comparison, copying, and length.
- ▶ Operations such as finding characters and substrings.

Character Class Testing and Conversion

`isalpha(c)`

`isupper(c)`

`islower(c)`

`isdigit(c)`

`isalnum(c)`

`isspace(c)`

`toupper(c)`

`tolower(c)`

- ▶ Functions from `<ctype.h>` for character testing and conversion.
- ▶ Check if a character is alphabetic, upper or lower case, digit, alphanumeric, blank, etc.

ungetc Function

```
int ungetc(int c, FILE *fp)
```

- ▶ A restricted version of `ungetch` that pushes a character back onto the file stream.
- ▶ Returns either the character (`c`) or EOF for an error.

Command Execution - system

```
int system(char *s)
```

- ▶ Executes the command contained in the character string s.
- ▶ Resumes execution of the current program after command execution.
- ▶ Returns a system-dependent integer status from the executed command.

Storage Management

```
void *malloc(size_t n)
void *calloc(size_t n, size_t size)
    // like malloc(n*size) with 0 initialisation
void free(void *p)
```

- ▶ Dynamic memory allocation functions.
- ▶ malloc returns a pointer to uninitialized storage.
- ▶ calloc returns a pointer to zero-initialized space for an array.
- ▶ free releases the dynamically allocated memory.

Mathematical Functions

```
sin(x)  
cos(x)  
atan2(y, x)  
exp(x)  
log(x)  
log10(x)  
pow(x, y)  
sqrt(x)  
fabs(x)
```

- ▶ A selection of frequently used mathematical functions from `<math.h>`.
- ▶ Operations include trigonometric, exponential, logarithmic, and power functions.

Random Number Generation

```
int rand()  
void srand(unsigned)  
#define frand() ((double) rand() / (RAND_MAX+1.0))
```

- ▶ rand computes a sequence of pseudo-random integers.
- ▶ srand sets the seed for rand.
- ▶ frand produces random floating-point numbers between 0 and 1.